

Arquitectura de Subagentes en Google Antigravity y Especificación Técnica de agents.yaml: Aislamiento de Contexto, Seguridad y Orquestación Declarativa¹. Fundamentos y Evolución de la Arquitectura de Agentes en Antigravity y ADK

La evolución de las arquitecturas de agentes basados en modelos de lenguaje de gran escala (LLM) ha transitado desde la ejecución imperativa de scripts hacia esquemas declarativos altamente estructurados y desacoplados. En el ecosistema de inteligencia artificial de Google, esta transición se consolida en la plataforma Google Antigravity 2.0 y su interfaz de línea de comandos reescrita en Go (antigravity-cli o agy), la cual sustituye la antigua arquitectura de Gemini CLI y comparte un arnés ejecutor unificado con la aplicación de escritorio de Antigravity. Paralelamente, el marco de trabajo Agent Development Kit (ADK) formaliza la separación entre la lógica de control del sistema y la configuración operativa mediante la funcionalidad Agent Config. A través del uso de esquemas de validación JSON estructurados (AgentConfig.json), los ingenieros de sistemas pueden definir el comportamiento, las herramientas y los límites normativos de un agente mediante archivos en formato YAML (agents.yaml o root_agent.yaml), evitando acoplar la orquestación directamente al código ejecutable. Esta separación declarativa responde a la necesidad de establecer fronteras normativas auditables y límites operativos infranqueables en entornos de desarrollo corporativo. La definición de agentes mediante archivos YAML delimita estáticamente el alcance del modelo, especificando explícitamente qué herramientas están autorizadas, qué subagentes pueden ser instanciados y qué restricciones de aislamiento contextual y del sistema de archivos deben imponerse durante el ciclo de vida de la tarea. La arquitectura de Antigravity admite además una orquestación jerárquica con capacidad de anidamiento recursivo hasta niveles de profundidad de nietos y descendientes posteriores, donde las actualizaciones de estado y subtrayectorias se transmiten recursivamente hacia la conversación raíz (root conversation) para mantener una visibilidad ejecutiva sobre el progreso global sin saturar la ventana de contexto del orquestador principal.

2. Especificación Sintáctica y Esquema de Definición: agents.yaml

El archivo de configuración agents.yaml (o root_agent.yaml en proyectos basados en ADK) constituye la especificación fundamental para la declaración de identidades, modelos, herramientas y subagentes asociados dentro del entorno de Antigravity y ADK. Cada documento YAML es validado en tiempo de carga mediante el esquema estandarizado AgentConfig.json, garantizando la consistencia sintáctica antes de la inicialización de los componentes en memoria.

Campo YAML	Tipo de Dato Requerido	Descripción Operativa y Arquitectónica
name	string	Sí Identificador único del agente en el espacio de nombres de la orquestación.
agent_class	string	No Clase base del agente (p. ej., LlmAgent). Define el motor conceptual subyacente en el runner de ADK.
model	string	Sí Alias o identificador del modelo LLM asignado (p. ej., gemini-2.5-flash, gemini-2.0-pro).
description	string	Sí Resumen ejecutivo del rol del agente utilizado por el

orquestador para delegar tareas.`instructionstring` Sí Instrucciones del sistema (System Prompt) que dirigen las reglas de razonamiento y comportamiento.`toolslistNoLista` explícita de herramientas locales o conjuntos de API autorizados para su uso.`disallowedToolslistNoLista` de herramientas bloqueadas categóricamente, anulando permisos implícitos.`sub_agentslistNoReferencias` relativas a archivos YAML de subagentes delegados (`config_path`).`isolation_modeenumNoNivel` de aislamiento de ejecución (`in-process`, `worktree`, `remote`).`permissionModeenumNoRégimen` de autorización operativa (`plan`, `default`, `acceptEdits`, `auto`, `dontAsk`, `bypassPermissions`).`effortstring/enumNoNivel` de esfuerzo computacional o presupuesto de razonamiento asignado a la ejecución.`maxTurnsintegerNoUmbral` máximo de iteraciones en el bucle ReAct antes de forzar una condición de paro.`mcpServerslist/objectNoConfiguración` de servidores del Protocolo de Contexto de Modelos (Model Context Protocol) adjuntos.`hooksobject/pathNoRuta` o configuración de puntos de intercepción previos y posteriores a las herramientas (`.agents/hooks.json`).`memory_scopeenumNoÁmbito` de visibilidad de la memoria persistente del agente (`project`, `user`, `local`). La declaración de la arquitectura se articula vinculando un agente orquestador principal con subagentes especializados alojados en el mismo módulo o en subdirectorios relativos. A continuación se presenta la configuración del agente raíz en `agents.yaml`:
YAML# `yaml-language-server`:
`$schema=https://raw.githubusercontent.com/google/adk-python/refs/heads/main/src/google/adk/agents/config_schemas/AgentConfig.json`

`agent_class`: `LlmAgent`

`name`: `orchestrator_agent`

`model`: `gemini-2.5-flash`

`description`: Agente principal de orquestación encargado de coordinar la arquitectura y delegar tareas de análisis.

`instruction`: |

Usted es el orquestador principal. Analice las solicitudes del usuario y determine si requieren

ejecución directa o delegación a subagentes especializados. Si se requiere modificar código,

delegue en el subagente `refactor_agent`.

`effort`: `high`

`permissionMode`: `default`

isolation_mode: in-process

maxTurns: 15

sub_agents:

- config_path: ./subagents/refactor_agent.yaml

tools:

- google_search
- read_file

hooks:

pre_tool_use: .agents/hooks.json

La definición del subagente especializado (./subagents/refactor_agent.yaml) refuerza las fronteras de ejecución al revocar el acceso a herramientas de red y restringir las modificaciones al sistema de archivos mediante un árbol de trabajo dedicado:YAML# yaml-language-server: \$schema=https://raw.githubusercontent.com/google/adk-python/refs/heads/main/src/google/adk/agents/config_schemas/AgentConfig.json

agent_class: LlmAgent

name: refactor_agent

model: gemini-2.5-flash

description: Subagente especializado en refactorización de código y corrección de sintaxis en entornos aislados.

instruction: |

Usted es un subagente de ingeniería de software. Su único objetivo es analizar y refactorizar código.

No intente realizar búsquedas en la red ni ejecutar comandos fuera del repositorio asignado.

isolation_mode: worktree

permissionMode: acceptEdits

maxTurns: 10

tools:

- read_file
- write_file
- run_tests

disallowedTools:

- execute_bash
- web_fetch

3. Mecanismos de Aislamiento de Contexto y Sidechains Transcripcionales La delegación de tareas en arquitecturas multiagente sin restricciones de contexto genera un incremento crítico en el consumo de tokens y una degradación acelerada de la atención del modelo. La ejecución no regulada de equipos de agentes en modos de planificación puede elevar el consumo de tokens en un factor aproximado de siete veces (7x) debido a la acumulación de trazas intermedias de razonamiento y resultados extensos de herramientas. Para resolver este problema, Antigravity implementa la arquitectura de transcripciones en cadena lateral (Sidechain Transcripts). Bajo este esquema, la actividad de un subagente no se registra directamente en el flujo de conversación principal del agente padre. En su lugar, el subagente escribe su historial de interacción en un par de archivos independientes compuestos por un registro persistente de eventos en formato JSONL (.jsonl), que almacena de manera secuencial los mensajes, llamadas a herramientas y snapshots del sistema, y un archivo de metadatos (.meta.json), que registra la configuración del agente y el estado de la tarea. Una vez concluida la labor del subagente, el arnés de Antigravity extrae exclusivamente un resumen de alto nivel (summary-only return) que se reinyecta en el hilo del agente orquestador, manteniendo la ventana de contexto del agente padre protegida contra la acumulación de datos intermedios. El sistema establece además un estricto contrato de invocación según el coste de contexto mediante la distinción entre SkillTool y AgentTool. La invocación de una SkillTool expande las instrucciones asociadas directamente sobre la ventana de conversación activa, ofreciendo una latencia mínima a expensas de consumir tokens del presupuesto principal. Por el contrario, la invocación de un AgentTool exige la instanciación de un nuevo subagente con su propia ventana de

memoria y su propia sidechain, una alternativa con mayor sobrecarga inicial pero indispensable para prevenir la degradación de la atención en tareas complejas. Previo a cada llamada al LLM, el motor de ejecución somete la ventana de contexto a un pipeline secuencial de cinco moldeadores (pre-model shapers) ordenados por su coste y destructividad informativa. En primer lugar, la etapa de Budget Reduction evalúa los límites de memoria y descarta elementos obsoletos. A continuación, la fase de Snip recorta respuestas antiguas de herramientas. Posteriormente, Microcompact elimina formateos redundantes y espacios vacíos en las trazas históricas. En cuarto lugar, Context Collapse realiza una proyección sintética no destructiva de la memoria en tiempo de lectura. Finalmente, la etapa de Auto-Compact aplica un resumen integral guiado por un modelo auxiliar una vez rebasado el umbral crítico, fijando un nuevo límite de compactación (compact boundary).

4. Matriz de Seguridad, Modos de Aislamiento e Intercepción de Herramientas

El control de las capacidades operativas de los subagentes exige una infraestructura de seguridad basada en múltiples capas de contención física y lógica. La especificación de Antigravity gestiona el aislamiento del entorno de ejecución mediante tres modalidades seleccionables en el archivo de configuración. La modalidad In-Process ejecuta al subagente en el mismo proceso del sistema pero manteniendo aislada su conversación, coordinando los accesos concurrentes al disco mediante bloqueos del sistema operativo (flock()). La modalidad Git Worktree crea un árbol de trabajo secundario dentro del repositorio Git, de modo que las modificaciones de código y ejecuciones de prueba ocurran en un directorio aislado sin alterar el espacio de trabajo principal del usuario. Por último, la modalidad Remote Execution delega la ejecución a contenedores o microVMs externas, aislando completamente los procesos del sistema de archivos local y de la red local del host.

La evaluación de autorizaciones para la invocación de herramientas se rige por un esquema de permisos de siete niveles y una política estricta de Deny-First, según la cual una regla de denegación siempre prevalece sobre cualquier permiso de autorización concedido previamente. El flujo de procesamiento de una solicitud de herramienta sigue una canalización secuencial determinista. En primera instancia, la petición se contrasta contra la lista estática `disallowedTools` para filtrar invocaciones prohibidas. Seguidamente, la solicitud pasa por los hooks de intercepción a nivel de espacio de trabajo definidos en `.agents/hooks.json`, donde scripts locales o funciones de validación evalúan el contexto de la llamada antes de autorizar su paso. Si la prueba es superada, el sistema aplica las reglas de denegación explícita y los límites del modo de permisos activo. En modos automatizados (auto), la llamada se somete además a un clasificador basado en aprendizaje automático (`yoloClassifier`) que realiza un análisis en dos fases (filtro rápido y cadena de pensamiento) para estimar el riesgo de la acción. Únicamente tras aprobar la totalidad de los controles, la orden es enviada para su ejecución dentro del entorno aislado configurado. Esta arquitectura de seguridad mitiga de

manera directa vulnerabilidades críticas detectadas en agentes de desarrollo, como la amenaza de GhostApproval (CVE-2026-12958). Esta vulnerabilidad permitía a repositorios booby-trapped engañar al usuario presentando en la interfaz de confirmación el nombre de un archivo inofensivo que en realidad era un enlace simbólico (symlink) apuntando a archivos sensibles como ~/.ssh/authorized_keys o ~/.zshrc. Antigravity neutraliza este vector forzando la resolución canónica de rutas físicas antes de desplegar el cuadro de diálogo de aprobación al usuario. Adicionalmente, el arnés resuelve el problema de la ventana de ejecución previa a la confianza (Pre-trust Execution Window), garantizando que ni los hooks ni los servidores MCP puedan ejecutarse durante la inicialización del proyecto sin que el usuario haya marcado explícitamente el directorio como seguro en ~/.gemini/config/.5. Orquestación Multiagente Recursiva y Topologías de DelegaciónLa arquitectura de Antigravity permite estructurar topologías de delegación complejas que superan los modelos tradicionales de consulta y respuesta. La capacidad de anidamiento recursivo habilita a los subagentes para actuar como coordinadores intermedios, instanciando sus propios subagentes descendientes según la complejidad del problema. El intercambio de información entre estos niveles se gestiona a través de la retransmisión ascendente de actualizaciones de subtrayectorias (subtrajectory updates), permitiendo que la conversación raíz conserve una representación clara del avance global de la tarea.

Vector ArquitectónicoGoogle Antigravity / ADKClaude CodeOpenClawSourcegraph AmpEspecificación de AgentesArchivos declarativos YAML (agents.yaml / AgentConfig).Frontmatter YAML en archivos Markdown (AGENTS.md).Manifiestos JSON/YAML con scripts BOOTSTRAP.md.Definición imperativa mediante código en el SDK.Aislamiento TranscripcionalCadenas laterales (Sidechains) .jsonl + .meta.json.Ficheros transcripcionales aislados con retorno sintético.Memoria compartida y enrutamiento por canales.Hilos de conversación compartidos y pasarela de mensajes.Aislamiento de EjecuciónPOSIX flock(), Git Worktree, MicroVMs.In-Process, Git Worktrees, Entornos Remotos.Canales distribuidos (Slack, Feishu, Telegram).Subprocesos locales y aislamiento a nivel de hilo.Evaluación de PermisosPolítica Deny-First con 7 modos + Hooks .agents/hooks.json.7 Modos de permisos + Clasificador ML de riesgo.Asignación de grupos y reglas de binding.Permisos contextuales basados en la sesión activa.Estructura de DelegaciónJerárquica Recursiva (Raíz-Hijo-Nieto).Delegación jerárquica con retornos condensados.Colaboración adversarial y enrutamiento.Red Peer-to-Peer mediante la herramienta agentToAgent.La comparación pone de relieve una convergencia en la industria hacia el uso de arquitecturas de contexto aislado mediante sidechains, reduciendo la sobrecarga de tokens en tareas prolongadas. Mientras que plataformas como OpenClaw priorizan la integración a través de canales de mensajería externos y Sourcegraph Amp adopta un patrón Peer-to-Peer, Google Antigravity enfoca su diseño en una orquestación jerárquica auditable guiada por contratos estáticos de configuración.6.

Conclusiones La arquitectura de subagentes de Google Antigravity y la especificación técnica de `agents.yaml` representan un modelo sólido para el desarrollo seguro y eficiente de sistemas autónomos de ingeniería de software. La adopción de esquemas declarativos en YAML respaldados por validación mediante `AgentConfig.json` proporciona a las organizaciones un mecanismo riguroso para auditar y gobernar las capacidades asignadas a cada componente antes de su ejecución. El uso de transcripciones en cadena lateral (sidechains) combinado con el retorno exclusivo de resúmenes sintéticos resuelve de manera efectiva el crecimiento descontrolado de la ventana de contexto, optimizando el consumo de tokens y protegiendo al agente orquestador de la degradación informativa. Al mismo tiempo, el soporte para entornos de ejecución aislados mediante Git Worktrees y la aplicación estricta de políticas de autorización Deny-First neutralizan vectores de ataque complejos como el abuso de enlaces simbólicos. La implementación coordinada de estas salvaguardas consolida un marco operativo adecuado para la integración de agentes de IA en entornos de producción de alta exigencia.